

Programming Language Principles, Design, and Implementation

Lecture 7 - The Simply Typed λ -Calculus

Sean Moss

University of Birmingham

Where are we?

- ▶ **Part 1: Principles of programming**
 - ▶ Functional languages
 - ▶ Operational semantics
 - ▶ **Type systems**
 - ▶ Abstract datatypes
- ▶ Part 2: Compilers

Today

The Simply-Typed λ -Calculus

- ▶ Motivation
- ▶ Syntax
- ▶ Typing rules
- ▶ Church-style vs. Curry-style

Further reading:

- ▶ “Types and Programming Languages”, Pierce, Chapter 9

Types

The Simply Typed λ -calculus: one of the most elementary type systems

Types are used to help explain the **meaning** of terms.

For example if we can assign the type $\mathbb{N}$ to an expression then we know that it computes to a number without evaluating it

Types:

- ▶ provide guarantees on how terms compute
- ▶ in particular that typed expressions **do not get stuck**
- ▶ document the code
- ▶ used for optimizations (e.g., memory usage)

Type checking is the process of checking that some given **expression** has some given **type** w.r.t. given **typing rules**

Therefore, **type checking** provides a lightweight **verification** technique

Syntax of the Simply Typed λ -Calculus

A typed variant of the untyped λ -calculus

- ▶ built on top of a collection of ground types.
- ▶ we consider a single ground type: **Booleans** ($\mathbb{B}$)
- ▶ the only other type is the type of functions (e.g., $\mathbb{B} \rightarrow \mathbb{B}$)

Syntax of simple types:

$$T ::= \mathbb{B} \mid T \rightarrow T$$

Syntax of terms:

$$M ::= x \mid \lambda x : T. M \mid M M \mid \text{true} \mid \text{false} \mid \text{if } M \text{ then } M \text{ else } M$$

Syntax of the Simply Typed λ -Calculus

Syntax:

$$T ::= \mathbb{B} \mid T \rightarrow T$$
$$M ::= x \mid \lambda x : T. M \mid M M \mid \text{true} \mid \text{false} \mid \text{if } M \text{ then } M \text{ else } M$$

Note:

- ▶ we add two Boolean constants and one operation to make the examples more interesting — these are primitives.
- ▶ a λ -abstraction $\lambda x : T. M$ mentions the type T of its argument — explicitly typed (Church-style)
 - ▶ to indicate the type of arguments functions expect
- ▶ convention: $\rightarrow$ is associative to the right (i.e., $T_1 \rightarrow T_2 \rightarrow T_3$ stands for $T_1 \rightarrow (T_2 \rightarrow T_3)$)

Examples:

- ▶ types: $\mathbb{B}, \mathbb{B} \rightarrow \mathbb{B}, (\mathbb{B} \rightarrow \mathbb{B}) \rightarrow \mathbb{B}, (\mathbb{B} \rightarrow \mathbb{B}) \rightarrow (\mathbb{B} \rightarrow \mathbb{B})$
- ▶ terms: $\lambda x : \mathbb{B}. \text{if } x \text{ then false else true},$
 $\lambda x : \mathbb{B}. \lambda y : \mathbb{B} \rightarrow \mathbb{B}. \text{if } x \text{ then } x \text{ else } y$

Small-step call-by-value operational semantics

Values:

$$V ::= \lambda x : T. M \mid \text{true} \mid \text{false}$$

Call-by-value evaluation contexts

$$C ::= \bullet \mid C \ M \mid V \ C \mid \text{if } C \text{ then } M \text{ else } M$$

Rules:

$$\frac{}{(\lambda x : T. M) \ V \longrightarrow_v M[x \setminus V]} \beta$$

$$\frac{}{\text{if true then } M \text{ else } N \longrightarrow_v M} \text{IteT}$$

$$\frac{}{\text{if false then } M \text{ else } N \longrightarrow_v N} \text{IteF}$$

$$\frac{M \longrightarrow_v N}{C[M] \longrightarrow_v C[N]} \text{CTX}_C$$

Typing

The Simply Typed λ -Calculus includes **typing rules** that associate types with expressions.

Derive **type judgements** of the form $\Gamma \vdash M : T$

where a **typing environment/context** Γ is a partial map from variables to types.

- ▶ Gives types to the free variables of M .
- ▶ Written in the form $x_1 : T_1, \dots, x_n : T_n$
- ▶ Maps each x_i to T_i (i.e., $\Gamma(x_i) = T_i$)
- ▶ We write $\cdot$ for the empty map
- ▶ $\text{dom}(\Gamma) = \{x_1, \dots, x_n\}$
- ▶ let $\Gamma, x : T$ be the context that
 - ▶ maps x to T
 - ▶ and each $y \in \text{dom}(\Gamma)$ such that $y \neq x$ to $\Gamma(y)$

Examples:

- ▶ an environment: $x : \mathbb{B}, y : \mathbb{B}$
- ▶ $x : \mathbb{B}, y : \mathbb{B}, x : \mathbb{B} \rightarrow \mathbb{B}$ is $y : \mathbb{B}, x : \mathbb{B} \rightarrow \mathbb{B}$

Typing Rules

Rules (syntax directed)

$$\frac{}{\Gamma, x : T \vdash x : T} \text{VAR}$$

$$\frac{\Gamma, x : T \vdash M : U}{\Gamma \vdash \lambda x : T. M : T \rightarrow U} \text{ABS}$$

$$\frac{\Gamma \vdash M : T \rightarrow U \quad \Gamma \vdash N : T}{\Gamma \vdash M N : U} \text{APP}$$

$$\frac{}{\Gamma \vdash \text{true} : \mathbb{B}} \text{T}$$

$$\frac{}{\Gamma \vdash \text{false} : \mathbb{B}} \text{F}$$

$$\frac{\Gamma \vdash M : \mathbb{B} \quad \Gamma \vdash N : T \quad \Gamma \vdash P : T}{\Gamma \vdash \text{if } M \text{ then } N \text{ else } P : T} \text{ITE}$$

Typing Rules

Prove that $(\lambda x : \mathbb{B}.x)\mathbf{true}$ has type $\mathbb{B}$

Proof:

$$\frac{\frac{\frac{}{x : \mathbb{B} \vdash x : \mathbb{B}} \text{VAR}}{\cdot \vdash \lambda x : \mathbb{B}.x : \mathbb{B} \rightarrow \mathbb{B}} \text{ABS} \quad \frac{}{\cdot \vdash \mathbf{true} : \mathbb{B}} \text{T}}{\cdot \vdash (\lambda x : \mathbb{B}.x)\mathbf{true} : \mathbb{B}} \text{APP}$$

Typing Rules

Using the above rules, what type can you assign to $\lambda x : \mathbb{B}. \lambda f : \mathbb{B} \rightarrow \mathbb{B} \rightarrow \mathbb{B}. f \ x \ x$?

Proof:

$$\begin{array}{c}
 \frac{}{x : \mathbb{B}, f : \mathbb{B} \rightarrow \mathbb{B} \rightarrow \mathbb{B} \vdash f : \mathbb{B} \rightarrow \mathbb{B} \rightarrow \mathbb{B}} \text{VAR} \quad \frac{}{x : \mathbb{B}, f : \mathbb{B} \rightarrow \mathbb{B} \rightarrow \mathbb{B} \vdash x : \mathbb{B}} \text{VAR} \\
 \frac{}{x : \mathbb{B}, f : \mathbb{B} \rightarrow \mathbb{B} \rightarrow \mathbb{B} \vdash f \ x : \mathbb{B} \rightarrow \mathbb{B}} \text{APP} \quad \frac{}{x : \mathbb{B}, f : \mathbb{B} \rightarrow \mathbb{B} \rightarrow \mathbb{B} \vdash x : \mathbb{B}} \text{VAR} \\
 \frac{}{x : \mathbb{B}, f : \mathbb{B} \rightarrow \mathbb{B} \rightarrow \mathbb{B} \vdash f \ x \ x : \mathbb{B}} \text{APP} \\
 \frac{}{x : \mathbb{B} \vdash \lambda f : \mathbb{B} \rightarrow \mathbb{B} \rightarrow \mathbb{B}. f \ x \ x : (\mathbb{B} \rightarrow \mathbb{B} \rightarrow \mathbb{B}) \rightarrow \mathbb{B}} \text{ABS} \\
 \frac{}{\vdash \lambda x : \mathbb{B}. \lambda f : \mathbb{B} \rightarrow \mathbb{B} \rightarrow \mathbb{B}. f \ x \ x : \mathbb{B} \rightarrow (\mathbb{B} \rightarrow \mathbb{B} \rightarrow \mathbb{B}) \rightarrow \mathbb{B}} \text{ABS}
 \end{array}$$

Typing Rules

In what context Γ can we assign the term $f (\lambda x : \mathbb{B}.x)$ **true** the type $\mathbb{B}$?

$$\Gamma = f : (\mathbb{B} \rightarrow \mathbb{B}) \rightarrow \mathbb{B} \rightarrow \mathbb{B}$$

What (other) types can we assign to $f (\lambda x : \mathbb{B}.x)$ **true** and in what contexts?

For any type T :

- ▶ we can assign it the type T
- ▶ in the context $\Gamma = f : (\mathbb{B} \rightarrow \mathbb{B}) \rightarrow \mathbb{B} \rightarrow T$

Expressive Power

“Typed expressions do not get stuck”

Can we find expressions that do not have types and get stuck?

Examples:

- ▶ `true true` does not have a type
- ▶ `if ( $\lambda x : \mathbb{B}.x$ ) then true else false`

Can we find expressions that do not have types and do not get stuck?

What about $\Delta\Delta$, where $\Delta = \lambda x : T.xx$?

What about $\Delta = \lambda x : T.xx$? What type can it be assigned?

$$\frac{\frac{x : T \vdash x : T \rightarrow U \quad x : T \vdash x : T}{x : T \vdash xx : U} \text{APP}}{\vdash \lambda x : T.xx : T \rightarrow U} \text{ABS}$$

We cannot complete this proof as x would have to have both type $T \rightarrow U$ and type T

Expressive Power

- ▶ This type system guarantees that typed expressions do not get stuck.
- ▶ It also prevents us from typing some expressions that do not get stuck.
- ▶ Not Turing-complete anymore.

Properties

Uniqueness: In a given context Γ , a term M (with free variables all in $\text{dom}(\Gamma)$) has at most one type, and one derivation that it has this type.

Canonical forms: A value of type $\mathbb{B}$ is either `true` or `false`, and a value of type $T \rightarrow U$ is of the form $\lambda x : T. M$.

Progress: A closed well-typed term M either is a value or it can be reduced further, i.e., there exists N , such that $M \rightarrow_v N$.

Substitution: If $\Gamma, x : U \vdash M : T$ and $\Gamma \vdash N : U$ then $\Gamma \vdash M[x \backslash N] : T$.

Preservation: If $\Gamma \vdash M : T$ and $M \rightarrow_v N$ then $\Gamma \vdash N : T$.

Safety = Progress + Preservation: Well-typed terms do not get stuck.

Properties

Uniqueness: In a given context Γ , a term M (with free variables all in $\text{dom}(\Gamma)$) has at most one type, and one derivation that it has this type.

Proof Sketch:

Assume $\Gamma \vdash M : T_1$ and $\Gamma \vdash M : T_2$ and prove $T_1 = T_2$.

By induction on M (or on the derivations):

- ▶ **VAR:** our assumptions are of the form $\Gamma, x : T \vdash x : T_1$ and $\Gamma, x : T \vdash x : T_2$, and by **VAR** $T_1 = T_2 = T$
- ▶ **ABS:** our assumptions are of the form $\Gamma \vdash \lambda x : U. N : T_1$ and $\Gamma \vdash \lambda x : U. N : T_2$, such that $T_1 = U \rightarrow U_1$, $T_2 = U \rightarrow U_2$, $\Gamma, x : U \vdash N : U_1$, and $\Gamma, x : U \vdash N : U_2$. By Induction Hypothesis (IH), $U_1 = U_2$, and therefore $T_1 = T_2$.
- ▶ **APP:** our assumptions are of the form $\Gamma \vdash M_1 N_1 : T_1$ and $\Gamma \vdash M_1 N_1 : T_2$, such that $\Gamma \vdash M_1 : U_1 \rightarrow T_1$, $\Gamma \vdash N_1 : U_1$, $\Gamma \vdash M_1 : U_2 \rightarrow T_2$, and $\Gamma \vdash N_1 : U_2$. By (IH), $U_1 \rightarrow T_1 = U_2 \rightarrow T_2$, and therefore $T_1 = T_2$.
- ▶ The other cases are similar.

Properties

Canonical forms: A value of type $\mathbb{B}$ is either `true` or `false`, and a value of type $T \rightarrow U$ is of the form $\lambda x : T.M$.

Proof Sketch:

(1) Assume that M is a value of type $\mathbb{B}$ and prove that M is either `true` or `false` (the other part of the theorem has a similar proof).

By cases on M (or on the derivation):

- ▶ VAR, APP, and ITE: variables are not values
- ▶ ABS: the function type $T \rightarrow U$ is not the $\mathbb{B}$ type
- ▶ T and F: the values are of the right form

Properties

Progress: A closed well-typed term M either is a value or it can be reduced further, i.e., there exists N , such that $M \rightarrow_v N$.

Proof Sketch:

Assume that $\Gamma \vdash M : T$ where M is closed, and prove that either M is a value or it can be reduced further.

By induction on M (or on the derivation):

- ▶ VAR: M is not closed
- ▶ ABS: M is a value
- ▶ APP: M is of the form $P N$, $\Gamma \vdash P : U \rightarrow T$, and $\Gamma \vdash N : U$ for some type U . By IH, either P is a value or it can be reduced further. If P is a value, then by **Canonical forms** property, P is of the form $\lambda x : U. Q$, and therefore $(\lambda x : U. Q)N$ reduces to $Q[x \backslash N]$. If P reduces further, then $P N$ also reduces further.
- ▶ T and F: M is a value
- ▶ ITE: similar to APP

Properties

The second part of **safety** requires proving **Substitution** and **Preservation**.

Substitution: If $\Gamma, x : U \vdash M : T$ and $\Gamma \vdash N : U$ then $\Gamma \vdash M[x \backslash N] : T$.

The proof is by induction on $\Gamma, x : U \vdash M : T$.

Preservation: If $\Gamma \vdash M : T$ and $M \longrightarrow_v N$ then $\Gamma \vdash N : T$.

The proof is by induction on $\Gamma \vdash M : T$ and makes use of **Substitution**.

We leave proving these as exercises

Church-style vs. Curry-style

λ -abstraction **explicitly** indicate the type of arguments they expect (Church-style): $\lambda x : T.M$.

We could instead try to deduce this type from the uses of x in M .

This results in another version of the Simply Typed λ -Calculus:

Curry-style.

Grammar of the Curry-Style Simply Typed λ -Calculus:

$$T ::= \mathbb{B} \mid T \rightarrow T$$
$$M ::= x \mid \lambda x.M \mid M M \mid \text{true} \mid \text{false} \mid \text{if } M \text{ then } M \text{ else } M$$

This has important consequences:

- ▶ **Uniqueness** is not true anymore.
- ▶ e.g., $\lambda x.x$ can be assigned any type of the form $T \rightarrow T$

Church-style vs. Curry-style

The only rule that differs is the λ -abstraction rule

We use $\Gamma \vdash_i M : T$ for this calculus.

Curry-Style Simply Typed λ -Calculus's λ -abstraction rule:

$$\frac{\Gamma, x : T \vdash_i M : U}{\Gamma \vdash_i \lambda x.M : T \rightarrow U} \text{ ABS}$$

The two calculi are related through an **erasure** function:

<code>erase(<i>x</i>)</code>	<code>=</code>	<code><i>x</i></code>
<code>erase($\lambda x : T.M$)</code>	<code>=</code>	<code>$\lambda x.$</code> erase(<i>M</i>)
<code>erase(<i>M N</i>)</code>	<code>=</code>	<code>erase(<i>M</i>) erase(<i>N</i>)</code>
<code>erase(true)</code>	<code>=</code>	<code>true</code>
<code>erase(false)</code>	<code>=</code>	<code>false</code>
<code>erase(if <i>M</i> then <i>N</i> else <i>P</i>)</code>	<code>=</code>	<code>if erase(<i>M</i>) then erase(<i>N</i>) else erase(<i>N</i>)</code>

Erasure property: if $\Gamma \vdash M : T$ then $\Gamma \vdash_i \text{erase}(M) : T$.

Conclusion

What did we cover today?

- ▶ The Simply Typed λ -calculus
- ▶ Motivation
- ▶ Syntax
- ▶ Typing rules
- ▶ Church-style vs. Curry-style

This week

- ▶ Thursday: Vincent will take you through Type Inference.
- ▶ Friday: tutorial/consolidation session on types.

Next week

- ▶ Advanced Types topics: polymorphism, abstract data types, existential types.

Week after next

- ▶ Consolidation week — revision/Q&A for all Programming Language topics.